

PythonBasicCourse-es000-IDE

June 21, 2026

1 Curso básico de Python

1.1 Apuntes

Curso básico de Python. Apuntes por Marcelo Horacio Fortino. Versión 2.5.1. Junio 2026.

Esta obra está sujeta a la licencia Reconocimiento-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-sa/4.0/>. Puede hallar permisos más allá de los concedidos con esta licencia en <https://fortinux.com>. Sugerencias y comentarios a info@fortinux.com.

Todas las marcas son propiedad de sus respectivos dueños. Python® y PyCon® son marcas registradas de la Python Software Foundation. Linux® es una marca registrada de Linus Torvalds. Ubuntu® es una marca registrada de Canonical Limited. Google® es una marca registrada de Google Inc. Microsoft® y Windows® son marcas registradas de Microsoft Corporation. Red Hat® es una marca comercial registradas de Red Hat, Inc. UNIX® es una marca comercial registrada de The Open Group. Apple® y Mac OS® son marcas comerciales de Apple Inc.

Versión	Autor/es	Fecha	Observaciones
1.0	Marcelo Horacio Fortino	2021/Marzo	Curso Python
1.1	Marcelo Horacio Fortino	2021/Junio	Convertido a markdown - ipynb
1.2	Marcelo Horacio Fortino	2021/Agosto	Actualizados contenidos
1.3	Marcelo Horacio Fortino	2021/Octubre	Agregado Flask microframework
1.4	Marcelo Horacio Fortino	2021/Noviembre	Agregado Pandas - datascience
1.5	Marcelo Horacio Fortino	2021/Diciembre	Agregado Devops - Ansible
2.0	Marcelo Horacio Fortino	2022/Abril	Nueva estructura: core / module
2.1	Marcelo Horacio Fortino	2022/Junio	Módulo apuntes intermedio
2.2	Marcelo Horacio Fortino	2022/Agosto	Actualizado temario y ejercicios
2.3	Marcelo Horacio Fortino	2022/Octubre	Actualizado despliegue a render.com

Versión	Autor/es	Fecha	Observaciones
2.4	Marcelo Horacio Fortino	2023/Noviembre	Aplicaciones de escritorio con python
2.5	Marcelo Horacio Fortino	2026/Abril	Aplicaciones de ML e IA para python

Esta obra se distribuye con la esperanza de que sea útil, pero SIN NINGUNA GARANTÍA, incluso sin la garantía MERCANTIL implícita o sin garantizar la CONVENIENCIA PARA UN PROPÓSITO PARTICULAR. El autor no asume ninguna responsabilidad si el lector hace un mal uso de la misma.

Sugerencias y comentarios a info@fortinux.com.

- Estos apuntes se basan en:
 - La documentación oficial de Python, <https://docs.python.org/es/3/tutorial/index.html>,
 - La bibliografía presentada al final de este documento, y
 - Documentación propia recogida a lo largo de los años de diversas fuentes.

1.2 Objetivo del curso

- Objetivo general del curso:
- Aprender a usar Python para crear scripts y programas simples plenamente funcionales, desarrollar aplicaciones web, realizar análisis de datos, y utilizar bibliotecas de inteligencia artificial (AI) para aprendizaje automático (ML).
- Objetivos específicos:
 - Reconocer las características principales de Python y su utilidad práctica.
 - Identificar tipos de datos (simples y compuestos) y operadores.
 - Aplicar variables y estructuras de control de flujo.
 - Construir funciones y clases (POO).
 - Clasificar los módulos y paquetes por sus funcionalidades y objetivos.
 - Comparar el lenguaje con otros similares de scripts, procedimentales y orientados a objetos.
 - Probar bibliotecas para conexiones REST a aplicaciones web y bases de datos.
 - Resolver problemas y errores en el código fuente proponiendo soluciones alternativas (refactoring).
 - Utilizar la biblioteca pandas junto con matplotlib y numpy para realizar análisis estadísticos de datos y gráficos.
 - Descubrir las herramientas en python de ML y DL (*Deep Learning*) para la inteligencia artificial.
- Como resultado práctico al final del curso cada estudiante habrá creado una aplicación web utilizando el microframework de Python Flask.

1.3 Temario

- Introducción, instalación y compilación
- Datos, expresiones y sentencias

- Variables y funciones, control de flujo
- Clases y objetos, herencia, polimorfismo
- Entradas y salidas con Python
- Gestión de módulos, paquetes y bibliotecas
- Servicios y programas en red, REST API
- Desarrollo de aplicaciones web con Flask
- Análisis de datos con pandas, matplotlib y numpy
- Aprendizaje automático (ML) e inteligencia artificial (AI) con Python
- Módulos opcionales
 - DevOps con Ansible
 - Plotting con Python
 - SDK de GCP para Python
 - SQL ejemplos con pandas
 - Docker y kubernetes con Python
 - Aplicaciones de escritorio con python

1.4 Bibliografía

- Downey, A., Elkner, J., Meyers, C. (2015). Aprenda a Pensar Como un Programador con Python.
Recuperado de <https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>
- Kent D. Lee. (2014). Python, Programming Fundamentals Second Edition.
- Marzal Varó, A., Gracia Luengo, I., García Sevilla, Pedro. (2014). Introducción a la programación con Python 3.
Recuperado de <http://repositori.uji.es/xmlui/handle/10234/102653>
- Miller, B., Ranum, D. Solución de problemas con algoritmos y estructuras de datos usando Python. Traducido por Mauricio Orozco-Alzate, Universidad Nacional de Colombia - Sede Manizales.
Recuperado de <https://runestone.academy/ns/books/published/pythoned/index.html?mode=browsing>
- Shaw, Z. A. (2016). Learn Python 3 the Hard Way.
Recuperado de <https://learnpythonthehardway.org/>
- Van Rossum, G. and the Python development team. (2020). Documentación de Python en español.
Recuperado de https://python-docs-es.readthedocs.io/_/downloads/es/pdf/pdf/
- McGregor, S. E. (2022). Practical Python data wrangling and data quality. O'Reilly Media.
- Chollet, F. (2017). Deep learning with Python. Manning Publications.
- Russell, S., & Norvig, P. (2019). Artificial intelligence: A modern approach. Pearson Higher Education.
- D2L.ai: Interactive Deep Learning Book with Multi-Framework Code, Math, and Discussions.
Recuperado de <https://d2l.ai/>

2 Instalación de python y configuración de IDEs

- Instalación de Python, creación de ambientes virtuales, La biblioteca virtualenv, VS Code, Ambiente virtual en GNU/Linux, Ambiente virtual en Windows, Usar el IDE, Cursor.

2.1 Instalación de Python

- Para programar se necesitan las siguientes herramientas:
 - Un editor para escribir el código.
 - Una consola para ejecutarlo.
 - Un depurador que permita ejecutar el código paso a paso.
- La instalación estándar de Python 3 contiene una aplicación llamada IDLE.
- IDLE por sus siglas en inglés significa Desarrollo Integrado y Entorno de Aprendizaje (*Integrated Development and Learning Environment*).
- En los sistemas operativos GNU/Linux como Debian o Ubuntu se instala y ejecuta respectivamente con:

```
[ ]: sudo apt-get install idle3
idle
```

- El lenguaje Python viene ya instalado en muchos sistemas operativos GNU/Linux como Debian o Ubuntu.
- En Windows® 10 se puede instalar desde la tienda de Microsoft® o se puede descargar e instalar manualmente: <https://www.python.org/downloads/windows/>.
- Tutorial en Internet: <https://techexpert.tips/es/windows-es/instalacion-del-entorno-virtual-de-python-en-windows>.
- La instalación de Python por lo general estará en C:\Python38 (cambiando el número de versión), aunque se puede cambiar esta ruta durante la instalación.
- Para añadir este directorio a la ruta en el prompt de DOS:

```
[ ]: set path=%path%;C:\python38
```

- Resolver ERROR: <https://recursospython.com/guias-y-manuales/python-no-se-reconoce-como-un-comando-interno-o-externo/>.

2.2 Creación de ambientes virtuales

- La instalación de Python incluye otras herramientas como:
 - *pip* para la instalación de bibliotecas, módulos y paquetes externos, y
 - *venv* para crear ambientes virtuales.
- Un entorno virtual Python sirve para aislar el intérprete Python, las bibliotecas y los scripts en un espacio determinado (carpeta o directorio).
- Esto permite trabajar en un proyecto evitando problemas generados por actualizaciones y/o de dependencias con el intérprete y las bibliotecas.
- También facilita la compilación y que pueda ser portado a otros sistemas.
- Python a partir de su versión 3.3 incluye un módulo, *venv*, que permite la creación de entornos virtuales.

- Fuentes:
 - <https://peps.python.org/pep-0405/>.
 - La guía oficial de instalación y uso de *pip* y ambientes virtuales:
 - [Installing packages using pip and virtual environments](#).
- Para crear el ambiente virtual en GNU/Linux utilizamos el siguiente comando:

```
[ ]: python3 -m venv ambiente_virtual/venv
source ambiente_virtual/venv/bin/activate
```

En Windows el comando se ejecuta de la siguiente manera:

```
[ ]: py -m venv ambiente_virtual
.\ambiente_virtual\Scripts\activate
```

- Este comando crea el directorio de destino (suele ser *.venv*) y el archivo *pyvenv.cfg* con una clave *home* que apunta a la instalación de Python desde la cual se ejecutó el comando.
- Crea un subdirectorio *bin* (o *Scripts* en Windows) con una copia o un enlace simbólico de los binarios Python dependiendo de la plataforma o argumentos utilizados al crear el entorno.
- Crea un subdirectorio (inicialmente vacío) *lib/pythonX.Y/site-packages* (*Lib\site-packages* en Windows).
- Fuente: <https://docs.python.org/es/3/library/venv.html>.
- Instalar un paquete dentro del ambiente virtual se realiza de la misma forma que en un sistema con Python y *pip* instalados:

```
[ ]: python3 -m pip install -U Flask           # Instala microframework Flask
python3 -m pip install -U Flask==2.0.0       # instala versión específica
python3 -m pip install --upgrade Flask       # actualiza Flask
```

- El fichero *requirements.txt* sirve para instalar módulos de la aplicación si ésta es cambiada de entorno (Sistema operativo, Servidor, etc.).
- Se puede crear manualmente e actualizarlo cuando es necesario escribiendo los nombres de los módulos a medida que se instalan.
- Como ejemplo, en GNU/Linux creamos el fichero y agregamos *Flask* en el mismo:

```
[ ]: touch requirements.txt
echo "Flask" > requirements.txt
```

- Posteriormente para instalar todos los módulos en el nuevo entorno se ejecutará:

```
[ ]: python3 -m pip install -r requirements.txt
```

- La siguiente opción (alternativa a la anterior y preferible) crea el fichero *requirements.txt* automáticamente.
- Se suele ejecutar una vez completo el desarrollo de la aplicación y previo a la migración al nuevo entorno.

```
[ ]: pip freeze > requirements.txt
```

2.3 La biblioteca virtualenv

- Si se está utilizando Python 3.3 o superior, la forma preferida de gestionar ambientes virtuales es mediante el módulo *venv*.
- De todas maneras se puede contar con la biblioteca *virtualenv* como herramienta para crear ambientes virtuales, la cual agrega funcionalidades extras; entre ellas:
 - soporte a Python 2.7, configuración, mantenimiento, duplicación, y
 - resolución de problemas (*troubleshooting*).
- Fuente: <https://virtualenv.pypa.io/en/latest/>
- Para instalarlo en Ubuntu:

```
[ ]: sudo apt install -y build-essential libssl-dev libffi-dev python3-dev
sudo apt install python3-pip
sudo apt install -y python3-venv
```

- Para realizar el mismo procedimiento en Windows 10®:
 - Descargar *pip* de <https://bootstrap.pypa.io/get-pip.py>
 - Guardarlo en el directorio donde está el fichero ejecutable de python.
- Luego abrimos la línea de comandos y escribimos:

```
[ ]: python get-pip.py
```

- Finalmente instalamos *virtualenv* y lo activamos:

```
[ ]: pip install virtualenv
virtualenv virtual_env
cd virtual_env\Scripts
activate.bat
```

- Por último es aconsejable tener instalado un sistema de control de versiones (CVS) como GIT.
- En Windows 10® se puede descargar desde <https://git-scm.com/download/win>.
- En GNU/Linux Ubuntu ya viene instalado, pudiéndose ver su versión con el comando:

```
[ ]: git --version
```

- Un listado oficial de los más relevantes proyectos de creación de paquetes:
- https://packaging.python.org/en/latest/key_projects/#virtualenv.

2.4 VS Code

- En Visual Studio Code un espacio de trabajo es una colección virtual de carpetas abiertas simultáneamente que se definen mediante el fichero *.code-workspace*.
- Al abrir este archivo, se abre automáticamente la colección de carpetas, denominado espacio de trabajo “multi-raíz”, es decir, se pueden utilizar varias carpetas en el mismo proyecto.
- Cuando no se define un espacio de trabajo, por ejemplo cuando no se necesita una estructura de carpetas, se puede crear una configuración que se guarda en el fichero *.vscode/settings.json*.

- Para crear un espacio de trabajo **workspace**:
- Abrir Menu -> file -> New window
- Abrir Menu -> file -> open folder
- Abrir Menu -> file -> save workspace as
- De esta forma se crea el fichero **.code-workspace**.
- Fuente: <https://stackoverflow.com/questions/44629890/what-is-a-workspace-in-visual-studio-code>.

2.5 Ambiente virtual en GNU/Linux

- Para crear un ambiente virtual en GNU/Linux:
- Ctrl + Shift + P: Python: Create Terminal

```
[ ]: python -m venv .venv
```

- Otra opción es: Ctrl + Shift + P: Python: Create Environment, y seleccionar **Venv**.
- para tener el ambiente virtual activado: (venv) ...

```
[ ]: source venv/bin/activate
```

- Para desactivar el ambiente virtual:

```
[ ]: deactivate
```

- Fuente: <https://stackoverflow.com/questions/54106071/how-can-i-set-up-a-virtual-environment-for-python-in-visual-studio-code>

2.6 Ambiente virtual en Windows

- Para activar la ejecución de ambientes virtuales en Windows:
- Abrir PowerShell con permisos de administrador y agregar la siguiente política:

```
[ ]: Set-ExecutionPolicy Unrestricted -Force
```

- Para crear un ambiente virtual (windows):

Ctrl + Shift + P: Python: Create Terminal

```
[ ]: python -m venv venv
```

- Otra opción es: Ctrl + Shift + P: Python: Create Environment, y seleccionar **Venv**.
- para tener el ambiente virtual activado: (venv) ...

```
[ ]: cd venv
.\Scripts\activate
```

2.7 Usar el IDE

- Ya se pueden instalar módulos, por ej.

```
[ ]: python -m pip install cowsay
```

- Se crea el fichero *mi_vaquita.py* con el siguiente código fuente:

```
[ ]: import cowsay

cowsay.cow('Hola mundo!')
```

-En la línea de comando o en *powershell* escribir:

```
[ ]: python mi_vaquita.py
```

Recursos: - https://code.visualstudio.com/docs/python/python-tutorial#_install-and-use-packages. - Python Tutorial for Beginners 1: Install and Setup for Mac and Windows - <https://www.youtube.com/watch?v=YYXdXT2l-Gg>.

- Python en windows:
- Configuración del entorno de desarrollo de Python en Windows
- <https://learn.microsoft.com/es-es/windows/dev-environment/python?tabs=winget>.
- Python extension for Visual Studio Code <https://marketplace.visualstudio.com/items?itemName=ms-python.python>
- <https://code.visualstudio.com/docs/python/environments>.
- <https://code.visualstudio.com/docs/languages/python>.
- <https://devblogs.microsoft.com/python/python-in-visual-studio-code-february-2026-release/>

2.8 Cursor

- Cursor 2.0 - Full Tutorial for Beginners
- <https://www.youtube.com/watch?v=l30Eb76Tk5s>.
- <https://cursor.com/for/python>.
- <https://cursor.fan/tutorial/HowTo/how-to-activate-virtualenv-in-cursor/>.
- Para poder ejecutar scripts, abrir PowerShell con permisos de administrador y agregar la siguiente política:

```
[ ]: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```